

Collecting Data from the Web

Concepts Every Researcher Should Know

Moses Boudourides

*Faculty, Graduate Program on Data Science
Northwestern University*

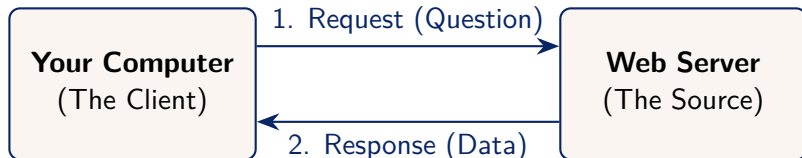
`Moses.Boudourides@northwestern.edu`

`Moses.Boudourides@gmail.com`

instats Seminar

Wednesday, May 13, 2026
4:00 PM – 7:00 PM UTC

The Big Picture: How Data Moves

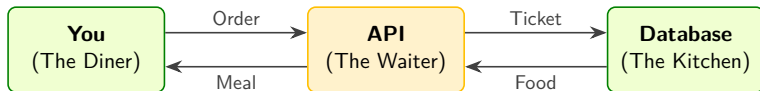


- **Client:** You (or your browser, or your script) asking for data.
- **Server:** A remote computer storing the database.
- **Request/Response:** The standard conversation format of the internet.

What is an API?

API = Application Programming Interface.

It is a set of rules allowing two computers to talk to each other.



- You cannot go into the kitchen and grab the food yourself.
- You must use the menu (the API documentation) to place a structured order.
- The waiter (the API) ensures your order is valid and brings exactly what you asked for.

HTTP Requests: GET vs POST

When you talk to an API, you use an HTTP method. The two most common are:

GET Request

Analogy: Asking a quick question.

- All your search criteria are visible in the URL.
- Used by most simple open APIs (WHO, ClinicalTrials).
- *Like searching on Google.*

POST Request

Analogy: Submitting a detailed form.

- The URL is plain. Search criteria are hidden in a "body" package.
- Used for complex, nested searches (NIH RePORTER).
- *Like filling out a tax return.*

Anatomy of a GET Request

A GET request puts all the search parameters directly into the web address.

```
https://clinicaltrials.gov/api/v2/studies?query.cond=diabetes&pageSize=50
```

- **Base URL:** The address of the API endpoint.
- **? (Question mark):** Separates the address from your questions.
- **Key=Value pair:** Your specific search criteria (e.g., condition is diabetes).
- **& (Ampersand):** Separates multiple questions.

Anatomy of a POST Request

A POST request keeps the URL clean and sends the details in a separate "body" package.

1. The plain URL:

`https://api.reporter.nih.gov/v2/projects/search`

2. The Request Body (sent invisibly alongside the URL):

```
{
  "criteria": {
    "advanced_text_search": {
      "search_text": "genomics"
    }
  },
  "limit": 50
}
```

What is JSON? The Spreadsheet Analogy

APIs return data in **JSON** (JavaScript Object Notation).
It is just a structured text file. You can read it like a spreadsheet.

Spreadsheet View

Name	Country	Year
Insulin	Human	2020
Albumin	Mouse	2021

Dictionary:

- **Spreadsheet** = [] (List)
- **Row** = { } (Record)
- **Cell** = "Key": "Value"

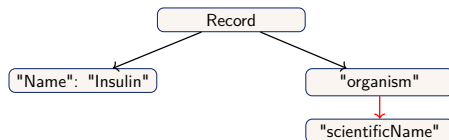
JSON View

```
[
  {
    "Name": "Insulin",
    "Country": "Human",
    "Year": 2020
  },
  {
    "Name": "Albumin",
    "Country": "Mouse",
    "Year": 2021
  }
]
```

Nested JSON and Dot Notation

Sometimes, a field contains another entire record inside it. This is called **nesting**.

```
{
  "Name": "Insulin",
  "organism": {
    "scientificName": "Homo sapiens",
    "commonName": "Human"
  }
}
```



To extract the scientific name, we use **dot notation**:

`organism.scientificName`

HTTP Status Codes: Did it work?

Every time you ask a server for data, it replies with a 3-digit status code.



Success!

Here is your data.



Client Error (Your fault)

404: Page not found.

401: You need an API key.

400: Your parameters are wrong.



Server Error (Their fault)

500: The server crashed.

503: The server is overloaded.

Try again later.

API vs Web Scraping: When to Use Which

If an API exists, **always use it**. If not, you must scrape.

Using an API

Invited: The website built a special door just for data requests.

Structured: Data arrives neatly formatted in JSON.

Stable: Rarely changes without warning.

Web Scraping

Uninvited: You are reading the public webpage meant for humans.

Messy: You must extract data from raw HTML code.

Fragile: Breaks instantly if the website redesigns its layout.

Web Scraping: Static vs Dynamic Tables

Not all tables on the internet can be easily scraped.

Static Tables (Scrapable)

What it is: The table data is written directly into the webpage's HTML code (using `<table>` tags).

Example: Wikipedia lists.

Why it works: When your script downloads the page, the data is already there.

Dynamic Tables (Hard)

What it is: The page loads empty, then uses JavaScript to fetch the data and draw the table.

Example: Interactive dashboards (Tableau, PowerBI).

Why it fails: Your script downloads the empty page before the JavaScript has time to run.

Summary & Key Takeaways

- **API:** A structured way to request data directly from a database.
- **GET vs POST:** GET puts search terms in the URL; POST puts them in a hidden body. Follow the API documentation.
- **JSON:** A structured text format that acts like a nested spreadsheet.
- **Dot Notation:** The way we navigate inside nested JSON records (e.g., `organism.scientificName`).
- **Web Scraping:** A fallback method to extract HTML tables when no API exists.
- **Static vs Dynamic:** Simple scrapers only work on static HTML tables, not JavaScript dashboards.

Further Reading & References

APIs and HTTP

- MDN Web Docs — *An Overview of HTTP*
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- MDN Web Docs — *HTTP Request Methods (GET, POST, ...)*
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- REST API Tutorial — *What is REST?*
<https://restfulapi.net/>

JSON

- MDN Web Docs — *Working with JSON*
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/
- W3Schools — *What is JSON?*
https://www.w3schools.com/whatis/whatis_json.asp

Web Scraping

- Real Python — *Web Scraping with Python: An Introduction*
<https://realpython.com/python-web-scraping-practical-introduction/>
- W3Schools — *HTML Tables (understanding the <table> structure)*
https://www.w3schools.com/html/html_tables.asp